Hafta 1 — Sunucu Tarafı Web ve ASP.NET Core'a Giriş

> **Ders:** Web Programlama · **Hafta:** 1 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** `Web Teknolojileri` dersi (HTML, CSS, JavaScript, HTTP'nin istemci yüzü) **Bu dosya sınav kapsamındadır.** Daha derin konular `H01-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Statik bir site ile dinamik bir web uygulaması arasındaki farkı **açıklayabileceksiniz**.
2. HTTP'nin sunucu tarafından nasıl göründüğünü — metotlar, durum kodları, başlıklar — **okuyabileceksiniz**.
3. HTTP'nin **durumsuz (stateless)** olmasının ne anlama geldiğini ve neden bir sorun olduğunu **anlatabileceksiniz**.
4. .NET Framework ile .NET (Core) arasındaki farkı ve **LTS** kavramını **ayırt edebileceksiniz**.
5. Bir ASP.NET Core MVC projesinin klasör yapısındaki her parçanın ne işe yaradığını **söyleyebileceksiniz**.
6. `Program.cs` dosyasını satır satır **okuyabileceksiniz**.
7. Bir isteğin tarayıcıdan veritabanına ve geri dönen yolculuğunu **şema üzerinde çizebileceksiniz**.
8. Bu zincirin hangi halkasının bu derse, hangilerinin başka derslere ait olduğunu **ayırt edebileceksiniz**.
9. Ters vekil ile uygulama sunucusu (Kestrel) arasındaki farkı **açıklayabileceksiniz**.

🧩 **Haftanın ilkesi — Katmanlama**

> Bir istek, her biri tek bir işten sorumlu katmanlardan geçer. Hiçbir katman diğerinin işini bilmek zorunda değildir — ama **zincirin tamamı kullanıcının gördüğü sonucu belirler.** Bu dersin işi belirli bir katmandır; onu bilmek kadar **sınırını bilmek** de gerekir.

---

1. Nereden Geliyoruz, Nereye Gidiyoruz?

`Web Teknolojileri` dersinde webin **istemci tarafını** öğrendiniz: tarayıcı bir adrese istek gönderiyor, karşılığında HTML alıyor, onu DOM'a çeviriyor, CSS ile biçimlendiriyor ve JavaScript ile etkileşim kazandırıyordu.

O dersin bilinçli olarak kara kutu bıraktığı bir yer vardı: **isteğin gittiği yer.**

```
[ Tarayıcı ] ─── HTTP isteği ──► [ ??? ]
[ Tarayıcı ] ◄── HTTP yanıtı ─── [ ??? ]
                                    ▲
                          İşte bu dersin konusu burası
```

Bu dönem o kutuyu açıyoruz. Dönem sonunda o kutunun içinde sizin yazdığınız kod olacak.

1.1. Bu ders neyi öğretir, neyi öğretmez

Bu derste	Bu derste **yok**	Nerede
Sunucuda çalışan C# kodu	HTML/CSS/JS öğretimi	`Web Teknolojileri` (ön koşul)
İsteğin sunucudaki yolculuğu	TCP/IP, DNS, TLS ayrıntısı	`Bilgisayar Ağları`
EF Core ile veritabanı erişimi	SQL, normalizasyon, indeks	`Veritabanı Yönetim Sistemleri`
Sunucu tarafı güvenlik (CSRF, yetkilendirme)	Kriptografi, saldırı sınıflandırması	`Bilgi Güvenliği`
Uygulamayı yayına almanın temeli	CI/CD, konteyner, ölçekleme	`Sistem Yöneticiliği`, `Altyapı`

HTML yazacağız — ama HTML öğretmeyeceğiz. Razor içinde HTML göreceksiniz ve bunu zaten bildiğiniz varsayılacak.

1.2. Bu ders nasıl işliyor

Bilmeniz gereken üç pratik nokta var:

① **Sınıfta bilgisayarınız yok — kodlama evde.** Derste hoca ekranda canlı kodlar, siz tahmin edersiniz, sonra birlikte çalıştırıp doğrularız. Yazma işi **haftalık ev görevine** devredilir. Bu ev görevleri rastgele alıştırmalar değildir: **her biri dönem projenizin bir parçasıdır.** On üç ev görevini yapan öğrenci, dönem sonunda projesini sıfırdan yazmaz — birleştirir.

② **Her hafta beş dosya alırsınız.** Hangisinin ne işe yaradığını bilmek zaman kazandırır:

Dosya	Ne zaman açılır
`-ders-notu.md`	**Bu dosya.** Derse gelmeden okuyun, sınava buradan çalışın
`-uygulama.md`	Derste yapılanların adım adım dökümü — evde tekrar ederken
`-detay.md`	Merak ettiğinizde. **Sınavda sorulmaz**, dosyanın başında yazar
`-etkilesimli.html`	Derste yansıtılan sayfa; kendiniz de açıp gezebilirsiniz
`-anlatim.md`	Hocanın hazırlık belgesi — sizin için değil

③ **Dönem boyunca tek bir uygulama büyür.** Her hafta aynı uygulamaya bir katman ekleriz: bu hafta iskelet, **Hafta 4**'te form, **Hafta 8**'de veritabanı, **Hafta 11**'de üyelik. Konular havada durmaz; eksik parçayı görürsünüz.

---

2. Statik Site ↔ Dinamik Uygulama

2.1. Statik site

Sunucuda hazır duran dosyalar vardır. İstek gelir, sunucu **dosyayı olduğu gibi** gönderir.

```
GET /hakkimizda.html
       |
       ▼
[ Sunucu ]  →  diskteki hakkimizda.html dosyasını oku  →  gönder
```

Herkese aynı içerik gider. `Web Teknolojileri` dersinde GitHub Pages'e koyduğunuz sayfa böyleydi.

2.2. Dinamik uygulama

Sunucuda **hazır dosya yoktur.** İstek gelir, sunucuda bir program çalışır, HTML'i **o anda üretir.**

```
GET /Urun/Detay/42
       |
       ▼
[ Sunucu ]  →  kodu çalıştır  →  veritabanından 42 numaralı ürünü çek
                              →  HTML'i üret
                              →  gönder
```

Aynı adres, farklı kullanıcıya **farklı içerik** döndürebilir. Sepetiniz sizin sepetinizdir.

2.3. Aradaki gerçek fark

Fark "dosya var mı yok mu" değil, **kararın nerede verildiğidir**:

	Statik	**Dinamik**
İçeriği kim üretir	Geliştirici, önceden	Sunucudaki kod, istek anında
Kullanıcıya göre değişir mi	Hayır	Evet
Veritabanı gerekir mi	Hayır	Genellikle evet
Kim çalıştırır	Hiç kimse — dosya okunur	Uygulama sunucusu

> 💡 **Ara biçimler de vardır.** Bazı siteler HTML'i önceden üretip statik gibi sunar (*static site generation*). Bu dersin konusu değil; bkz. `H01-detay.md`.

---

3. HTTP — Sunucu Tarafından Görünüşü

HTTP'yi `Web Teknolojileri`'nde istemci gözüyle gördünüz. Şimdi **yanıtı üreten taraf** sizsiniz.

3.1. Bir isteğin ham hâli

```
POST /Uye/Giris HTTP/1.1
Host: kampuskitap.local
Content-Type: application/x-www-form-urlencoded
Content-Length: 41
User-Agent: Mozilla/5.0 ...
Accept: text/html,application/xhtml+xml
Cookie: .AspNetCore.Session=CfDJ8N...

eposta=ayse@example.com&parola=gizli123
```

Dört bölüm vardır:

1. **İstek satırı** — metot (`POST`), yol (`/Uye/Giris`), sürüm (`HTTP/1.1`)
2. **Başlıklar** — her satır bir `Ad: Değer` çifti
3. **Boş satır** — başlıkların bittiğini gösterir
4. **Gövde (body)** — yalnızca bazı metotlarda bulunur

Sunucudaki kodunuz bunların **hepsine** erişebilir. Dönem boyunca bu parçaları tek tek kullanacağız: `Cookie` başlığını **Hafta 7**'de, gövdeyi **Hafta 4**'te okuyacağız.

Sunucu tarafında işinize yarayacak başlıklar

Onlarca başlık vardır; bu dersin ilgilendikleri şunlardır:

Başlık	Yön	Ne işe yarar
`Host`	İstek	Hangi siteyi istediğiniz. Tek sunucuda çok site barındırmayı mümkün kılar
`Content-Type`	İkisi	Gövdenin biçimi (`application/json`, `application/x-www-form-urlencoded`). **Hafta 12**'de API'de yanlış olursa 415 alırsınız
`Content-Length`	İkisi	Gövdenin bayt uzunluğu
`Cookie`	İstek	İstemcinin sakladığı değerler — **Hafta 7**
`Set-Cookie`	Yanıt	Sunucunun istemciye "şunu sakla" demesi — **Hafta 7**
`Location`	Yanıt	Yönlendirmede gidilecek adres; `302` ile birlikte kullanılır
`Accept`	İstek	İstemcinin kabul ettiği biçimler — API'de HTML mi JSON mu döneceğine karar verir
`Authorization`	İstek	Kimlik bilgisi taşır — **Hafta 11-12**

> ⚠️ **Altın kural:** Başlıkların tamamı **istemciden gelir** ve istemci onları istediği gibi değiştirebilir. `User-Agent` yalan söyleyebilir, `Content-Length` yanlış olabilir. **Hiçbir güvenlik kararını istemciden gelen bir başlığa dayandırmayın.** Bu cümleyi **Hafta 4, 6 ve 11**'de tekrar duyacaksınız.

3.2. Metotlar

Metot	Anlamı	Gövdesi var mı	Bu derste ilk nerede

GET	Getir, okuma amaçlı	Hayır	Hafta 2
POST	Yeni bir şey oluştur / işlem yap	Evet	Hafta 4
PUT	Var olanı tamamen değiştir	Evet	Hafta 13
PATCH	Var olanın bir kısmını değiştir	Evet	Hafta 13
DELETE	Sil	Genellikle hayır	Hafta 12

İki kavram sınavda ve mülakatta sorulur:

- **Güvenli (safe) metot:** Sunucuda bir şey değiştirmez. `GET` güvenlidir.
- **Etkisiz (idempotent) metot:** Aynı isteği 1 kez de 10 kez de göndersen sonuç aynıdır. `GET`, `PUT`, `DELETE` etkisizdir. `POST` **değildir** — bu yüzden formu iki kez gönderince iki kayıt oluşur (**Hafta 4**'te bunu `PRG` deseniyle çözeceğiz).

3.3. Durum kodları

Yanıtın ilk satırındaki üç haneli sayı, **ne olduğunu tek bakışta** söyler.

Aralık	Anlamı	Bu derste kullanacaklarımız
2xx	Başarılı	`200 OK`, `201 Created`, `204 No Content`
3xx	Yönlendirme	`302 Found`, `301 Moved Permanently`
4xx	**İstemci** hatalı	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`
5xx	**Sunucu** hatalı	`500 Internal Server Error`

Ezberlenecek tek kural şudur:

> **4xx = "senin isteğinde sorun var" · 5xx = "benim kodumda sorun var"**

Bu ayrım önemlidir: 500 gördüğünüzde hata **sizin kodunuzdadır** ve loglara bakmanız gerekir. 404 gördüğünüzde adres yanlıştır.

İki sık karıştırılan çift:

- **401 ↔ 403:** 401 = "kim olduğunu bilmiyorum, giriş yap". 403 = "kim olduğunu biliyorum, ama bu senin işin değil." (Hafta 11)
- **200 ↔ 204:** İkisi de başarılı. 204 = başarılı ama **gövde yok**. (Hafta 11)

3.4. Stateless (durumsuzluk) — dönemin en önemli kavramı

HTTP stateless'tir (durumsuzdur): Her istek bağımsızdır. Sunucu, iki isteği kendiliğinden birbirine bağlamaz.

Şunu düşünün: Bir siteye giriş yaptınız. Sonra bir ürün sayfasına tıkladınız.

```
1. istek:  POST /Giris      →  "Merhaba Ayşe, giriş başarılı"
2. istek:  GET  /Urun/42    →  "Siz kimsiniz?"
```

Sunucu ikinci istekte sizi **hatırlamaz.** Çünkü HTTP'de hatırlama diye bir şey yoktur. Her istek, sunucunun ilk kez gördüğü bir istektir.

> 🧠 **Doğru zihin modeli:** Sunucu, her seferinde hafızası silinmiş bir memur gibidir. Onu tanıdığınıza ikna etmek için **her istekte kimliğinizi yeniden göstermeniz** gerekir.

Bu bir tasarım kusuru değil, **bilinçli bir tercihtir**: stateless olmak sayesinde istekler farklı sunuculara dağıtılabilir, sunucu çökse bile diğeri devam edebilir, ölçeklemek kolaydır.

Peki sepet nasıl çalışıyor? Çünkü **durumu biz elle taşıyoruz.** Üç yol vardır ve üçünü de bu dönem öğreneceğiz:

Yol	Nerede tutulur	Hafta
Çerez (Cookie)	İstemcide; her istekte otomatik gider	7
Oturum (Session)	Sunucuda; istemcide yalnızca anahtarı durur	7
Veritabanı	Sunucuda, kalıcı	10

Bu hafta çözmüyoruz — sorunu tanıyoruz. Hafta 7'nin tamamı bu soruna ayrılmıştır.

---

4. Bir İsteğin Uçtan Uca Yolculuğu

`Enter`'a bastınız. Ekranda sayfa belirene kadar arada ne oldu? Bu bölüm o zinciri baştan sona gösterir — ve **bu dersin o zincirin neresinde durduğunu** netleştirir.

> 📌 Bu anlatının kaynağı, konuyu adım adım açan açık kaynaklı bir belgedir: https://github.com/alex/what-happens-when. Türkçe uyarlaması ders klasöründeki `what-happens-when.html` dosyasıdır.

4.1. Zincirin tamamı

```
   1            2              3              4               5
[Tarayıcı] → [Ağ katmanı] → [Ters vekil] → [ASP.NET Core] → [Veritabanı]
                            └──────────── BU DERSİN BÖLGESİ ────────────┘

   ön koşul    Bilgisayar          bu dersin konusu
   dersler     Ağları dersi
```

Beş durak var. **Yalnızca son üçü bizim.** İlk ikisi başka derslerin konusu — ama zincirin parçası oldukları için bilmeniz gerekir.

4.2. Ağ katmanı — bu dersin konusu değil

Tarayıcı adresi çözdükten sonra şunlar olur:

Adım	Ne yapar	Hangi ders
DNS	Alan adını IP adresine çevirir	`Bilgisayar Ağları`

TCP	İki uç arasında bağlantı kurar (üç adımlı el sıkışma)	`Bilgisayar Ağları`
TLS	Bağlantıyı şifreler, sertifikayı doğrular	`Bilgisayar Ağları`

> ⚠️ **Bu üç adımı bu derste açmıyoruz.** Bilmeniz gereken tek şey: **zaman alırlar ve siz onları hızlandıramazsınız.** Sayfanız yavaşsa suçlu her zaman sizin kodunuz değildir.

4.3. URL'nin anatomisi — sunucunun ne gördüğü

Bu, zincirin **bizi ilgilendiren** ilk parçasıdır:

```
https://kampuskitap.local:5001/Urun/Detay/42?renk=mavi
└──┘   └──────────┬──────────┘ └─┘└─────┬─────┘└──┬──┘
şema          host          port    yol      sorgu dizesi
```

Sunucuya **yol** ve **sorgu dizesi** gider. Şema ve host bağlantıyı kurmak için kullanılır.

Hafta 2'de bu yolun bir C# metoduna nasıl bağlandığını (routing), **Hafta 4**'te sorgu dizesinin ve form verisinin bir nesneye nasıl dönüştüğünü (Model Binding) göreceğiz.

4.4. İstek sunucuya vardı: modern sunucu mimarisi

Üretimde bir web uygulaması genellikle **iki katmanlıdır**:

```
İnternet
   │
   ▼
┌──────────────────────────────────┐
│ Ters vekil (reverse proxy)       │  IIS / Nginx / Apache
│ · TLS sonlandırma                │
│ · yük dengeleme                  │
│ · statik dosya önbelleği         │
└──────────────┬───────────────────┘
               ▼
┌──────────────────────────────────┐
│ Kestrel + ASP.NET Core uygulaması│  ← sizin kodunuz
└──────────────┬───────────────────┘
               ▼
          [ Veritabanı ]
```

Kestrel, ASP.NET Core uygulamanızın içinde gelen web sunucusudur; çapraz platformdur ve hızlıdır.

Neden iki katman? Ters vekil şunları ücretsiz verir: sertifika yönetimi, tek porttan birden çok uygulama sunma, yavaş istemci saldırılarına karşı koruma, uygulama çökerse yeniden başlatma.

> 💡 **Geliştirme makinenizde ters vekil yoktur.** `dotnet run` dediğinizde yalnızca Kestrel çalışır ve siz doğrudan ona bağlanırsınız. "*Yerelde çalışıyor, sunucuda çalışmıyor*" hatalarının bir kısmı bu katman farkından doğar — **Hafta 13**'te yayına alırken göreceğiz.

4.5. Yanıt geri dönerken

Yanıtınız kullanıcıya varmadan önce birkaç yerde saklanabilir:

Katman	Kimin için saklar
Ters vekil	Aynı dosyayı isteyen herkes için
CDN	Kullanıcıya coğrafi olarak yakın kopya
Tarayıcı	Aynı kullanıcının bir sonraki isteği için

Hangisinin ne kadar saklayacağını **sizin gönderdiğiniz yanıt başlıkları** belirler (`Cache-Control`, `ETag`).

> ⚠️ **Önbellek, bu dersin en sinsi hata kaynaklarından biridir.** "*Kodu değiştirdim ama sayfa eskisini gösteriyor*" şikâyetinin cevabı çoğu zaman buradadır. İlk bakılacak yer: Ctrl + F5 ve DevTools'taki **Disable cache** seçeneği.

4.6. HTTP sürümleri — kodunuzu neden etkilemiyor

	HTTP/1.1	HTTP/2	HTTP/3
Biçim	Düz metin	İkili	İkili
Taşıma	TCP	TCP	QUIC (UDP üzerinde)
Aynı anda çok istek	Sıra beklenir	Çoğullama	Çoğullama

Yazdığınız controller üç sürümde de aynıdır. ASP.NET Core farkı sizin yerinize halleder. Ayrıntı `H01-detay.md` §5'te.

4.7. Bu yolculuktan aklımızda kalması gerekenler

1. **Zincirin yalnızca üç halkası bizim** — ama kullanıcının gördüğü yanıtın tamamı bizim sorumluluğumuzda.
2. **Her halka gecikme ekler.** Kodunuz hızlı olsa da site yavaş olabilir.
3. **Sunucu, isteği ham metin olarak alır ve ham metin döndürür.** Aradaki her şey bizim ürettiğimizdir.

5. .NET Ekosistemi

5.1. İki ayrı .NET vardı

	.NET Framework	.NET (Core)
Çıkış	2002	2016
Çalıştığı yer	**Yalnızca Windows**	Windows, Linux, macOS

Kaynak kodu	Kapalı	**Açık kaynak**
Durumu	Yalnızca bakım; yeni özellik yok	**Aktif geliştirilen sürüm**
Web çatısı	ASP.NET (WebForms, MVC 5)	**ASP.NET Core**

Microsoft 2016'da .NET'i **sıfırdan yeniden yazdı.** Sebebi şuydu: .NET Framework Windows'a ve IIS'e sıkı sıkıya bağlıydı; bulut çağında bu sürdürülemezdi.

5.2. İsimlendirme neden karışık

Bu, öğrencilerin en çok takıldığı yerdir:

```
.NET Framework 1.0 … 4.8     ← eski dünya, yalnızca Windows
.NET Core 1.0 … 3.1          ← yeni dünya, "Core" ekiyle
.NET 5, 6, 7, 8, 9, 10       ← "Core" kelimesi düştü, aynı yeni dünya
```

.NET 5'ten itibaren **"Core" kelimesi atıldı** ve 4'ten sonra 5 gelince karışıklık olmasın diye numara atlandı. Yani:

> **.NET 10 = .NET Core'un devamıdır, .NET Framework 4.8'in devamı değildir.**

Ama web çatısının adı hâlâ **ASP.NET Core**'dur. Bu isim korundu.

5.3. LTS nedir, neden umurumuzda?

Microsoft her Kasım'da yeni bir .NET sürümü yayımlar. İki tür vardır:

Tür	Destek süresi	Ne zaman seçilir
LTS (Long Term Support)	3 yıl	Üretim, uzun ömürlü projeler, **dersler**
STS (Standard Term Support)	18 ay	Yeni özelliği hemen denemek isteyenler

Bu derste .NET 10 (LTS) kullanıyoruz — Kasım 2028'e kadar destekli.

> ⚠️ **.NET 8 kullanmayın.** .NET 8 ve .NET 9'un desteği **10 Kasım 2026'da sona eriyor.** Desteği biten sürüm güvenlik yaması almaz.

5.4. Dokümantasyon sürümü tuzağı

Bu dersin en çok zaman kaybettiren pratik detayı budur:

Microsoft dokümantasyonu **her sayfayı birden çok .NET sürümü için** yayımlar. Adresin sonundaki `?view=aspnetcore-10.0` parçası, hangi sürümün gösterileceğini belirler.

Arama motorundan gelen sonuç çoğu zaman **eski sürüme** düşer. Şu belirtiyi ezberleyin:

> **Örnekte `Startup.cs` görüyorsanız, yanlış sürümdesiniz.** .NET 6'dan beri o dosya yoktur (§5.3).

Kontrol listesi:

1. Sayfanın üstündeki sürüm seçicisinden **.NET 10**'u seçin, veya

2. Adresin sonuna elle `?view=aspnetcore-10.0` ekleyin.

---

6. İlk Proje: İskeletin Turu

Proje şu komutla oluşturulur:

```
dotnet new mvc -n IlkProje
```

> 📌 Kurulum ve ilk çalıştırma adımları `00-kurulum-rehberi.md` dosyasındadır — bu haftanın ev görevidir. Burada **oluşan projeyi tanıyoruz.**

6.1. Klasör yapısı

```
IlkProje/
├── Controllers/            ← istekleri karşılayan C# sınıfları
│   └── HomeController.cs
├── Models/                 ← veri sınıfları
│   └── ErrorViewModel.cs
├── Views/                  ← HTML üreten Razor şablonları
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   └── Shared/
│       └── _Layout.cshtml  ← ortak sayfa iskeleti
├── wwwroot/                ← DIŞARIYA AÇIK statik dosyalar
│   ├── css/
│   ├── js/
│   └── lib/
├── Properties/
│   └── launchSettings.json ← yalnızca geliştirme ayarları (port vb.)
├── appsettings.json        ← yapılandırma
├── Program.cs              ← uygulamanın başladığı yer
└── IlkProje.csproj         ← proje tanımı, paketler, hedef sürüm
```

Üç kritik nokta:

`wwwroot` özeldir. Bu klasördeki her şey **tarayıcıdan doğrudan erişilebilir.** `wwwroot/css/site.css` dosyasına `https://siteniz/css/site.css` adresiyle ulaşılır. Dışarıdan görülmemesi gereken hiçbir şeyi buraya koymayın.

`Controllers`, `Views`, `Models` isimleri sözleşmedir. ASP.NET Core bu isimleri arar. `Controllers` klasörünü `Denetleyiciler` diye adlandırırsanız çatı onu bulamaz.

`launchSettings.json` yalnızca sizin makinenizde geçerlidir. Yayına alındığında kullanılmaz. Port numarası burada yazar.

6.2. `Program.cs` — satır satır

MVC şablonunun ürettiği dosya tam olarak şudur:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

Dosya **dört evreye** ayrılır. Bu ayrımı görürseniz dosya bir daha karışık gelmez:

Evre	Satır	Ne oluyor
1. Kurucu	`CreateBuilder`	Yapılandırma okunur, temel servisler hazırlanır
2. Servis kaydı	`builder.Services.Add...`	"Uygulamada şu yetenekler olsun" denir
3. İnşa	`builder.Build()`	Uygulama nesnesi üretilir — **bu satırdan sonra servis eklenemez**
4. Pipeline	`app.Use...` / `app.Map...`	İstekleri kimin, hangi sırayla işleyeceği belirlenir

Son satır `app.Run()`, uygulamayı başlatır ve **istekleri dinlemeye** geçer.

> ⚠️ **En sık yapılan yeni başlayan hatası:** `builder.Build()` çağrıldıktan sonra `builder.Services.Add...` yazmak. Derlenir ama **hiçbir etkisi olmaz.** Servisler 2. evrede kaydedilir.

6.3. `Startup.cs` nereye gitti?

.NET 5 ve öncesinde iki dosya vardı: `Program.cs` (uygulamayı başlatır) ve `Startup.cs` (servisleri ve pipeline'ı kurardı). `Startup.cs` içinde iki metot bulunurdu: `ConfigureServices` ve `Configure`.

.NET 6'dan itibaren bu ikisi `Program.cs` **içinde birleştirildi.** Buna *minimal hosting model* denir. Yaptıkları iş aynıdır, yalnızca tek dosyada toplanmıştır:

Eski (.NET 5 ve öncesi)	Yeni (.NET 6+)
`Startup.ConfigureServices(...)`	`builder.Services.Add...`
`Startup.Configure(...)`	`app.Use...` / `app.Map...`

Neden bilmeniz gerekiyor: İnternetteki öğreticilerin çoğu hâlâ eski biçimdedir. `Startup.cs` gören öğrenci "bende bu dosya yok, yanlış bir şey yaptım" diye düşünür. Yapmadınız — öğretici eskidir.

6.4. Projeyi çalıştırmak ve `dotnet watch`

İki komut vardır ve farkları ev görevinizde işinize yarayacak:

```
dotnet run     # derle ve çalıştır
dotnet watch   # derle, çalıştır ve DEĞİŞİKLİKLERİ İZLE
```

`dotnet run` ile çalıştırdığınızda, kodda bir değişiklik yaptığınızda uygulamayı **durdurup yeniden başlatmanız** gerekir: Ctrl + C → `dotnet run`.

`dotnet watch` bunu sizin yerinize yapar: dosyaları izler, siz kaydettiğiniz anda değişikliği uygulamaya yansıtır (*hot reload*) — çoğu durumda tarayıcıyı bile yeniler. Görünüm (`.cshtml`) değişikliklerinde yeniden başlatma bile gerekmez.

> 💡 **Pratik tavsiye:** Geliştirirken `dotnet watch`, "çalışıyor mu" diye bakarken `dotnet run` kullanın. Bazı değişiklikler (örneğin `Program.cs`'e servis eklemek) hot reload ile yansımaz; o zaman `watch` size "yeniden başlatılsın mı?" diye sorar.

Uygulamayı durdurmak her iki durumda da Ctrl + C'dir. Durdurmadan ikinci bir kez çalıştırmaya kalkarsanız `Address already in use` hatasını alırsınız — bu haftanın Hata Köşesi'ndeki ikinci satır.

6.5. NuGet

NuGet, .NET'in **paket yöneticisidir** (JavaScript'teki `npm`'in karşılığı).

- Paketler `nuget.org` adresinden gelir.
- Eklenen her paket `.csproj` dosyasına bir `<PackageReference>` satırı olarak yazılır.
- Komut: `dotnet add package <PaketAdı>`

Bu dönem EF Core (Hafta 8) ve Identity (Hafta 11) paketlerini böyle ekleyeceğiz.

---

7. Microsoft Dokümantasyonunu Okumak

Bu ders `learn.microsoft.com` üzerinden ilerliyor. Dokümantasyon bir ders kitabı değil, **başvuru kaynağıdır** — baştan sona okunmaz, aranır. Birkaç pratik alışkanlık dönem boyunca zaman kazandırır:

① **Önce sürümü sabitleyin.** Adresin sonunda `?view=aspnetcore-10.0` yoksa ekleyin (§5.4).

② **Sayfa türünü tanıyın.** Üç tür vardır ve amaçları farklıdır:

Tür	Ne verir	Ne zaman açılır
Tutorial	Baştan sona çalışan bir örnek	Konuyu ilk kez öğrenirken
Conceptual / Overview	Kavramın ne olduğu ve neden var olduğu	"Bu ne işe yarıyor?" derken
API reference	Sınıf ve metot imzaları	Bir metodun aşırı yüklemesini ararken

Bu derste ağırlıklı olarak **tutorial** ve **conceptual** sayfalar kullanılacak. API reference'a yalnızca gerektiğinde bakılır.

③ **Kod bloklarının üstündeki sekmelere dikkat edin.** Çoğu sayfa aynı işi **Visual Studio / VS Code / .NET CLI** için ayrı ayrı anlatır. Yanlış sekmedeki komutu kopyalamak en sık yapılan hatadır.

④ **Ders notu ile dokümantasyon çelişirse.** Ders notu, dokümantasyonun **günlük kullanılan alt kümesidir**; sınav kapsamı ders notudur. Dokümantasyon daha geniştir ve daha fazla kenar durum içerir — ama ikisi çelişiyorsa (sürüm farkı dışında) bunu bildirin.

> 📌 **Bu haftanın okuma ödevi:** *Get started with ASP.NET Core MVC*, yalnızca **1. bölüm**. Seri 10 bölümdür ve dersin H1–H6 ile H10 akışını neredeyse birebir izler; her hafta ilgili bölümü okuyacaksınız.

🐛 Hata Köşesi — Hafta 1

Bu haftanın tipik hataları. Her hafta bu tabloya bir satır eklenecek; dönem sonunda elinizde 13 satırlık bir tanılama rehberi olacak.

Belirti	Gerçek sebep	Çözüm
`'dotnet' is not recognized`	Terminal kurulumdan önce açılmış; `PATH` güncel değil	Terminali kapat-aç, olmazsa yeniden başlat
`Address already in use` veya port hatası	Proje zaten çalışıyor ya da port başka programda	Önceki terminalde **Ctrl+C**; ya da `launchSettings.json`'dan portu değiştir
Tarayıcı "Bağlantınız gizli değil" diyor	Geliştirme sertifikasına güvenilmemiş	`dotnet dev-certs https --trust`
Sayfa açılmıyor, terminalde hata yok	Yanlış adrese bakılıyor	Terminaldeki `Now listening on:` satırındaki adresi kullan
Servis eklendi ama "servis bulunamadı" hatası	`builder.Build()`'dan **sonra** kaydedilmiş	Servis kaydını `Build()`'dan önceye taşı
Kodu değiştirdim, tarayıcı eskisini gösteriyor	Önbellek (tarayıcı veya ters vekil)	Ctrl + F5; DevTools → *Disable cache*
CSS/JS dosyası 404 veriyor	Dosya `wwwroot` dışında	`wwwroot/css/...` altına taşı
Öğreticideki `Startup.cs` bende yok	Öğretici .NET 5 ve öncesine ait	Normaldir; sürümü `?view=aspnetcore-10.0` yap

---

📖 Terimler Sözlüğü

Terim	Türkçe	Anlamı
SDK	Yazılım geliştirme kiti	Kod yazmak, derlemek ve çalıştırmak için gereken araçlar
Runtime	Çalışma zamanı	Derlenmiş uygulamayı çalıştıran bileşen
LTS	Uzun dönem destek	3 yıl desteklenen .NET sürümü
Stateless	Durumsuz	Her isteğin bağımsız olması; sunucunun hatırlamaması
NuGet	—	.NET paket yöneticisi
wwwroot	—	Dışarıya açık statik dosya klasörü
Minimal hosting	—	.NET 6+ ile gelen, `Startup.cs` siz tek dosyalı başlatma modeli
CDN	İçerik dağıtım ağı	İçeriği kullanıcıya yakın sunuculardan sunan ara katman
Reverse proxy	Ters vekil	Uygulamanın önünde duran, TLS ve yük dengeleme yapan sunucu
Kestrel	—	ASP.NET Core'un içinde gelen web sunucusu
Idempotent	Etkisiz	Aynı isteğin tekrarının sonucu değiştirmemesi

---

✅ Kontrol Soruları

> Bu sorular sınav soru havuzunun bir parçasıdır. Kod **okuma** ve **hata bulma** ağırlıklıdır — sınavın fiilî biçimiyle aynı.

A. Statik / dinamik ve HTTP

1. Bir haber sitesinin ana sayfası ile bir e-posta kutusunun gelen kutusu: hangisi statik, hangisi dinamik olarak sunulmalıdır? Gerekçelendirin.
2. `GET` ile `POST` arasındaki iki temel farkı yazın.
3. Bir form iki kez gönderildiğinde iki kayıt oluşuyor. Bu, hangi metodun hangi özelliğiyle ilgilidir?
4. Aşağıdaki durum kodları hangi durumlarda döner? `201`, `204`, `401`, `403`, `500`
5. Kullanıcı bir sayfaya girdi ve `403` aldı. Giriş yapmış mıdır? Neden?
6. Bir istekte `500` alıyorsunuz. Hata sizin kodunuzda mı, isteği gönderenin kodunda mı?
7. “HTTP stateless’tir” cümlesini, bir alışveriş sepeti örneği üzerinden üç cümleyle açıklayın.
8. Stateless olmak bir kusur olsaydı çözülürdü. Stateless olmanın sağladığı **iki avantajı** yazın.

B. .NET ekosistemi

9. .NET Framework 4.8 ile .NET 10 arasındaki üç farkı yazın.
10. ".NET 10, .NET Framework 4.8'in devamıdır." Bu cümle doğru mu? Değilse düzeltin.
11. LTS ile STS arasındaki fark nedir? Bu derste neden LTS seçildi?
12. Bir öğretici sayfasında `Startup.cs` dosyası görüyorsunuz. Bu size ne söyler, ne yaparsınız?

13. `dotnet --version` komutu `8.0.404` çıktısı veriyor. Sorun var mı? Varsa nedir?

C. Proje iskeleti ve `Program.cs`

14. Aşağıdaki dosyaların her biri ne işe yarar? `Program.cs`, `appsettings.json`, `launchSettings.json`, `IlkProje.csproj`
15. `wwwroot` klasörünü diğerlerinden ayıran özellik nedir? Buraya **koymamanız gereken** bir dosyaya örnek verin.
16. `Controllers` klasörünün adını `Denetleyiciler` yaparsanız ne olur? Neden?
17. `Program.cs`'in dört evresini sırayla yazın ve her birinde ne olduğunu bir cümleyle anlatın.
18. Aşağıdaki kodda bir hata var. Bulun ve düzeltin:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
builder.Services.AddControllersWithViews();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

19. `.csproj` dosyasında `<PackageReference>` satırı ne anlama gelir? Oraya nasıl eklenir?

D. İsteğin yolculuğu

20. Bir isteğin geçtiği beş durağı sırayla yazın. Hangileri bu dersin konusudur?
21. DNS, TCP ve TLS adımları bu derste neden açılmıyor? Hangi derse aittir?
22. Aşağıdaki URL'de sunucuya **hangi parçalar gider**, hangileri gitmez?
    `https://kampuskitap.local:5001/Urun/Detay/42?renk=mavi`
23. Ters vekil (reverse proxy) ile Kestrel arasındaki farkı iki cümleyle yazın.
24. `dotnet run` dediğinizde yukarıdaki şemadaki hangi kutular çalışır, hangileri çalışmaz?
25. Ters vekilin sağladığı dört yarardan ikisini yazın.
26. "Kodu değiştirdim ama tarayıcı eski sayfayı gösteriyor." İlk üç şüpheliniz nedir?
27. HTTP/2'ye geçmek yazdığınız controller kodunu değiştirir mi? Neden?
28. Bir sayfa yavaş açılıyor. Suçlunun **sizin kodunuz olmadığı** üç senaryo yazın.

📑 Kaynakça

> Bu bölüm iki işe yarar: **(1)** merak ettiğiniz konuyu kaynağından okuyabilirsiniz, **(2)** bu ders notundaki her iddianın nereye dayandığı görünür olur. Microsoft bağlantılarındaki `?view=aspnetcore-10.0` parçasını silmeyin (§5.4). *Bağlantılar 18 Eylül 2026'da denetlendi.*

Bu haftanın zorunlu okuması

#	Kaynak	Neyi dayandırıyor
1	Get started with ASP.NET Core MVC — Part 1	§6 — proje oluşturma ve çalıştırma. **Bu haftanın okuma ödevi.** Serinin tamamı 10 bölümdür ve dersin H1–H6 + H10 akışını izler

HTTP (§3)

#	Kaynak	Neyi dayandırıyor
2	MDN — HTTP genel bakış	§3.1 isteğin dört bölümü, başlıkların yapısı
3	MDN — HTTP metotları	§3.2 metot tablosu, *güvenli* ve *etkisiz* tanımları
4	MDN — HTTP durum kodları	§3.3 durum kodu tablosu, 401 ↔ 403 ve 200 ↔ 204 ayrımları
5	RFC 9110 — HTTP Semantics	Protokolün **resmî tanımı.** MDN'in kaynağı budur; tartışmalı bir durumda buraya bakılır

> 💡 MDN sayfalarının çoğunun **Türkçe çevirisi** de vardır: adresteki `en-US` yerine `tr` yazın (örn. `developer.mozilla.org/tr/docs/Web/HTTP/Reference/Methods`). Çeviriler bazen eksik kalır; şüphede kalırsanız İngilizce sürüme bakın.

.NET ekosistemi ve sürümler (§4)

#	Kaynak	Neyi dayandırıyor
6	.NET resmî destek politikası	§4.3 — LTS/STS ayrımı, 3 yıl ↔ 18 ay, .NET 10'un Kasım 2028'e kadar destekli olması
7	.NET 8 ve .NET 9 destek sonu duyurusu	§4.3 — **10 Kasım 2026** tarihi. "Neden .NET 8 kurmuyoruz" sorusunun cevabı
8	.NET 5'ten 6'ya geçiş rehberi	§6.3 — `Startup.cs` 'in nasıl `Program.cs` 'e taşındığı, eski/yeni karşılığı
9	Minimal hosting modeli	§5.2 — `WebApplication.CreateBuilder` ve dört evre

Proje ve araçlar (§5)

#	Kaynak	Neyi dayandırıyor
10	`dotnet new` komutu	§5 — `dotnet new mvc` ve şablon seçenekleri
11	`dotnet watch` komutu	§6.4 — hot reload, hangi değişikliklerin yeniden başlatma gerektirdiği
12	NuGet nedir	§6.5 — paket yöneticisi, `PackageReference`
13	**MVC şablonunun `Program.cs` kaynağı**	§5.2 — ders notundaki `Program.cs` **birebir bu dosyadan** alınmıştır, elle yazılmamıştır. `#if` satırları şablon koşullarıdır; kimlik doğrulamasız seçenekte kalan kısım bizim gördüğümüzdür

İsteğin yolculuğu (§4)

#	Kaynak	Neyi dayandırıyor
14	**what-happens-when**	§4 — bu bölümün tamamının kaynağı. Adres çubuğundan piksele kadar her adımı açan açık kaynaklı belge. **Ağ katmanı adımları da burada**; merak edenler oradan okuyabilir
15	MDN — HTTP önbellekleme	§4.5 — önbellek katmanları, `Cache-Control`, `ETag`
16	Kestrel web sunucusu	§4.4 — Kestrel nedir, ters vekil arkasında nasıl çalışır
17	ASP.NET Core'u barındırmak	§4.4 — ters vekil + uygulama sunucusu mimarisi (ayrıntısı Hafta 13)

Nasıl kullanılır

- **Sınava çalışırken** kaynakçaya gerek yok; sınav kapsamı bu ders notudur.
- **Bir şey anlamadıysanız** ilgili satırdaki kaynağa gidin — ders notu o kaynağın günlük kullanılan alt kümesidir (§7).
- **Ders notunda bir hata bulursanız** kaynağa bakıp bildirin; not düzeltilir.
- **Projenizde** bu kaynaklar birincil başvurunuzdur.

📚 Daha Fazlası

Aşağıdaki konular `H01-detay.md` dosyasındadır ve **sınav kapsamı dışıdır**:

- Webin sunucu tarafının tarihi: CGI'dan uygulama sunucusuna
- ASP.NET WebForms neden bırakıldı, `ViewState` neydi
- Kestrel, IIS ve ters vekil (reverse proxy) ilişkisi
- HTTP/1.1, HTTP/2 ve HTTP/3 farkları
- Diğer sunucu çatıları: Spring Boot, Django, Laravel, Express — karşılaştırma
- `WebApplication.CreateBuilder` arka planda tam olarak ne yapar
- Ortam değişkenleri ve yapılandırma sağlayıcılarının önceliği
- Statik dosya sunumunda `MapStaticAssets` ile `UseStaticFiles` farkı
- HTTP/1.1, HTTP/2 ve HTTP/3'ün ayrıntılı karşılaştırması

🔜 Haftaya Hazırlık

🏠 **Ev görevi (bu hafta):** `00-kurulum-rehberi.md` dosyasını izleyerek .NET 10 SDK ve bir IDE kurun, `dotnet new mvc` ile proje oluşturup çalıştırın. **İki ekran görüntüsü teslim edin:** `dotnet --version` çıktısı ve tarayıcıda açılan proje sayfası.

Okuma: Get started with ASP.NET Core MVC — Part 1 (yalnızca 1. bölüm; adresin sonundaki `?view=aspnetcore-10.0` parçasını silmeyin)

Haftaya (Hafta 2 — MVC Temelleri ve Mimari Yapı): `/Urun/Detay/42` adresine bir istek geldiğinde sunucunun bunu hangi C# metoduna bağladığını göreceğiz. Ayrıca bu hafta gördüğünüz `Program.cs`'teki `app.Use…` **satırlarının ne yaptığını** açacağız — istek, o satırların oluşturduğu bir zincirden geçiyor. Bu hafta kurduğumuz pipeline'daki `UseRouting` satırının içini açacağız.

Düşünerek gelin: `dotnet new mvc` ile oluşturduğunuz projede adres çubuğuna yalnızca `https://localhost:7xxx/` yazdığınızda bir sayfa açılıyor. Hiçbir adres belirtmediğiniz hâlde sunucu hangi kodu çalıştıracağını nereden bildi?